A unified lighting system for Hither

ENGINEERING PROPOSAL / 7 SEPTEMBER 2026

Recommendation: retain Bevy's clustered forward renderer, make every world surface participate in its lighting, and introduce a reusable LightEmitter component that turns an item's light value into actual illumination. Add a shared shadow budget, coherent sun and sky, and spatial ambient lighting. Treat dynamic global illumination as a later, measured quality upgrade.

The change that matters most

A glowing apple, carried lantern, or placed torch should light the ground, surrounding rocks, foliage, nearby characters, and the player's hands with the same color and falloff. The item author should assign a value and optional preset; the renderer should handle movement, shadows, visibility, streaming, and cleanup.

Today, terrain and rocks compute their own fixed lighting while most meshes use Bevy PBR. Fixing this split is the highest-confidence improvement. Bevy already has clustered point/spot lighting and a supported pattern for feeding custom material properties into PBR. Reuse those paths. Bevy 0.19.1 PBR shader; Bevy 0.19.1 material extension.

Priority	Proposed result
1. Correctness	Every material receives item lights; walls block important lights; no stale lights after item removal.
2. Art quality	Consistent roughness, exposure and daylight; warm local pools; readable caves with darker unlit spaces.
3. Performance	Finite influence ranges, spatial admission, shared shadow limits, preserved mesh sharing and surface caches.
4. Authoring	One light value, sensible presets, optional color/range/shadow overrides.

What "best" means here

Optimize for image quality per millisecond on Hither's existing desktop renderer. No single algorithm maximizes quality and performance simultaneously. The recommended baseline avoids a new full-scene geometry representation and makes the game's existing lights work consistently. Its actual cost must be measured; this proposal does not promise an FPS gain.

Scope and confidence

This is a proposal for implementation, based on the live working tree, locally installed Bevy 0.19.1 source, primary technical sources, existing screenshots, and the project's benchmark documentation. No game code was changed and no new runtime benchmark was performed. Existing screenshots are historical references, not verified captures of the current executable. Target hardware and FPS requirements remain unspecified; proposed budgets are starting points.

What the code actually does

The current code takes precedence over older README and performance notes. In particular, the project name still says SDF, but the active terrain and rock paths are rasterized; raymarch currently returns no surface.

Finding	Evidence in the working tree	Implication
Split lighting	assets/shaders/sdf_scene.wgsl:244, 995-1024	Terrain/rock color uses a fixed sun direction, ambient floor, specular exponent and rim term. Only directional shadow maps are sampled.
Separate world rig	src/player/avatar.rs:298-329	Global ambient 350, sun 8,500 lux and a second 1,800-lux directional fill are owned by avatar setup.
Independent shadow rules	src/world/goblin_dens/mod.rs:682-743 ; src/world/orcs/torch.rs:407-486	Dens choose four nearby shadowed lamps. Each torch enables its primary shadow inside 18 m; the code does not enforce its “nearest torch” comment.
Hand lighting differs	src/player/hand.rs:42-100	Camera-space hands on layer 2 have two fixed directional fills. Normal world lights do not automatically light them correctly.
HDR already exists	src/rendering/scene.rs:21-39; src/rendering/graphics.rs:125-170	World and hand passes compose before a final TonyMcMapface tone map. This should be preserved and calibrated.
Useful optimizations exist	surface_cache.rs, plant_lod.rs, radial_shadows.rs; grass.rs	Reuse cached material properties, shared meshes, simplified tree casters, depth prepass, GPU occlusion and grass displacement.

Additional causes of inconsistent appearance

The sky’s sun direction is separately hard-coded. The terrain’s old ambient-occlusion function samples map_scene near the origin, while map_scene is now only a ground plane; this is not scene-wide cave or foliage occlusion. Dens add a 100,000-lumen, 65 m unshadowed fill. These are explicit code facts; their relative visual impact requires a current capture.

Implications for the rework

Move world lighting ownership into rendering/lighting. Preserve the procedural material detail and collision geometry. Replace the fixed surface-light calculation, consolidate local-light policy, and replace broad interior fill with a spatial solution. Avoid designing a new SDF light tracer around an inactive scene representation.

The historical RTX 5090 audits establish useful testing infrastructure, not today’s baseline. They include earlier castle scenes and differing rendering configurations. Capture fresh matched cases before using them to judge the lighting rework. See docs/RENDER_PERFORMANCE_AUDIT.md and tools/benchmark_render.py.

Assign a light value to any item

Proposed API, not implemented code. The minimal form accepts luminous power in lumens, with a neutral omnidirectional preset. Presets supply a useful emission anchor, visible glow, range policy and animation.

```
commands.entity(item).insert(LightEmitter::new(800.0));

// Optional controls for an authored item:
LightEmitter::new(800.0)
    .preset(EmissionPreset::WarmLantern)
    .color(Color::srgb(1.0, 0.62, 0.24))
    .offset(Vec3::new(0.0, 0.25, 0.0));

// Serialized item definition, proposed:
"light": { "lumens": 800, "preset": "warm_lantern" }
```

One control, two coordinated outputs

The component creates an actual Bevy light and configures the designated luminous part of the item. Visible glow is not a substitute for illuminating neighboring objects. A preset maps the authored power to an appropriate surface glow; it does not equate surface brightness with total light power. PointLight intensity is lumens, and range is a finite cutoff. Bevy 0.19.1 point lights.

Default to one point light per compact object. Use a spot preset for a directional lamp. Large luminous objects may opt into a bounded arrangement of lights. Do not create one light per triangle or make an entire lantern handle emissive. Glow-mesh selection is optional; items without one still illuminate their surroundings.

Range and tuning

Derive a default influence radius from a configurable minimum contribution, then clamp it to the preset's maximum. For an ideal isotropic source, r = sqrt(lumens / (4*pi*minimum_lux)) is an initial estimate, not an exact perceptual cutoff. Preserve Bevy's attenuation and tune the cutoff for a smooth disappearance. Exposure and scene brightness affect the appropriate threshold. Advanced users may override range and source radius.

Lifecycle contract

Adding or enabling the component creates exactly one owned light child. Changing it updates that child; setting power to zero, disabling it, or removing the component removes its contribution. Despawning the item removes the child. Use item-local anchors and propagated transforms so equip, drop, reparent, animation and scale have defined behavior. World-space range stays in metres.

Inventory contents emit only when the item state explicitly permits it; equipped and placed items default to active. Existing gameplay needs an adapter for these states. Do not mutate a shared material asset when changing one item: use cached material variants or an instance parameter so other copies retain their appearance.

Use Changed/Added/Removed events for static emitters; animate only active flickering ones. Keep the light registry bounded to streamed content, and protect queued work with entity/generation checks. Clamp negative/non-finite values. New serialized fields need defaults. Nothing here requires one CPU update per inactive item per frame.

Make every surface respond consistently

Keep surface generation; replace illumination

Retain terrain snow, footprints, geology, rock contacts and distance-filtered surface caches. Separate their outputs into linear base color, geometric normal, shading normal, roughness and occlusion. Cached values remain material properties, so moving a light never invalidates a terrain material cache.

For the existing custom terrain/rock Material, first build PbrInput using Bevy's version-pinned helpers and call apply_pbr_lighting. Populate world position, fragment coordinates, view direction, both normals, material fields and shadow-receiver flags. A default PbrInput alone is insufficient. Use the actual world render target's pixel coordinates for clusters, not display-resolution UI coordinates. Bevy 0.19.1 PBR shader.

Keep this adapter small. ExtendedMaterial<StandardMaterial, ...> is the preferred pattern for new procedural materials, but immediately converting the existing terrain binding layout and vertex outputs is unnecessary scope. Bevy's official example demonstrates changing material inputs before shared lighting and post-lighting processing. Bevy 0.19.1 material extension.

Color and depth are one contract

Treat procedural colors as linear consistently with texture decoding. Apply roughness-based PBR instead of the fixed specular exponent and unconditional rim brightness. Keep footprint relief as material/occlusion detail; do not add an unexplained bright term after lighting. Avoid double exposure, per-material tone mapping and output clamping. Filament provides the physical-unit and linear-light reference. Filament rendering reference.

Audit terrain and rock depth/shadow variants explicitly. Camera-depth and shadow-map projections are different views. Portal cutouts and displaced fallback terrain must agree with visible geometry. Camera-distance discards must not remove a necessary shadow caster. The sky remains non-shadow-casting. Preserve reverse-Z depth, conservative sky depth, MSAA coverage and GPU culling bounds.

First-person hands and held emitters

Put the hand camera and hand rig into matching world-space transforms while retaining their separate depth buffer. This allows the hand material to use world-space light positions. Then validate light/render layers and shadow-map reuse across both views. A camera-space emitter proxy must not become the physical world light; the held item has one world-space emission anchor.

World geometry may need inclusion in the light's caster layers even though it is excluded from the hand camera. Measure whether Bevy duplicates shadow work for the second view. Replace fixed studio fills with shared environmental light and, if needed, a small explicitly artistic readability term that fades in darkness. Reconcile exposure on both cameras.

Finish the image coherently

Use one sun-direction/color resource for both the sky and light. Calibrate neutral surfaces, snow, skin and torch power at fixed exposure first. Preserve final HDR composition; add restrained bloom once after world/hand composition, ahead of UI. Bloom supplies the optical halo, not bounce illumination. Defer automatic exposure and global fog until they can cover all relevant passes consistently.

Bound the expensive work

Clustered shading reduces the local lights considered for a fragment; it does not make overlapping lights or shadow rendering free. Use Bevy's existing clusters and add a game-level admission and shadow policy above them.

One shadow scheduler for the whole game

Replace den and torch selection with a common scheduler. Rank lights by gameplay importance, estimated screen contribution and distance to the active world view. Add hysteresis and a minimum residency time so small camera movements do not swap shadow ownership. A held torch and a nearby light separated from the player by a wall are high priority.

A point shadow map has six faces. The point-light map resolution is controlled by a shared resource, so different per-light resolutions or a cached atlas are not free configuration switches. Start with native maps and one global resolution per quality preset. Do not promise persistent cached shadow updates without a separate render-pipeline change. Bevy 0.19.1 point lights.

Initial tuning values	Low	Balanced	High
Active local-light admission cap	64	128	256
Shadowed point-light slots	1	2	4
Point shadow face resolution	512	1024	1024
Sun cascades	2	2	2
Contact shadows / PCSS	Off	Off initially	Optional measured test

These are proposed test settings, not measured safe limits. Counts include all local-light producers; spots consume a corresponding shadow-view budget. Stress tests must vary overlap as well as total count. Keep existing sun resolution controls initially, and separate local-shadow settings from the current directional-only quality setting.

Budget overflow must not create glowing walls

Unshadowed lights can illuminate through solid geometry. A shadow slot eviction must therefore not blindly convert an important cave light to an unshadowed light. Reserve slots for consequential sources; fade or suppress lower-priority direct contribution if safe visibility cannot be maintained. Tiny unshadowed details are allowed only within conservative open-room placement/range limits. Full room/portal masking across arbitrary surfaces is extra shader work, not something RenderLayers solve per pixel.

Use light-influence bounds, not visibility of the glowing mesh, for admission: an offscreen lamp can illuminate an onscreen wall. Include any necessary offscreen casters in streaming coverage. Retain porous tree shadow proxies, make them available to local lights where appropriate, and keep grass from becoming a new dense shadow workload.

Torch consolidation

Test replacing four moving torch lobes with one primary light whose power/color follow the flame, with one optional small accent on High. Keep visible flame animation. Reduce intensity-only changes without changing transform when possible, but never claim that this automatically caches native shadow maps. Preserve the current appearance until side-by-side captures justify consolidation.

Ambient light, bounce, and alternatives

Direct-light consistency is the foundation. For the finished look, replace broad cave fill with spatial ambient illumination and conservative local bounce. Dark interiors and bright exterior entrances must coexist in the same view; camera-only ambient dimming cannot provide that.

Recommended second stage: bounded spatial probes

Prototype room-sized irradiance volumes for generated dens, with daylight visibility and low-frequency static bounce computed during bounded background generation. Bevy has an ambient-cube volume sampling path and documents its texture layout, but does not provide the baker. A procedural baker is new work. Hardware support and interpolation across walls require explicit validation. Bevy 0.19.1 irradiance volumes.

Start with one representative den and a static collision/mesh snapshot plus an acceleration structure. Use deterministic ray samples, reject solid probes, isolate disconnected rooms, and avoid interpolating through thin partitions. Schedule only a fixed number of jobs/uploads and cancel stale work. Cache by seed, geometry version and static-light revision. Moving items continue to provide immediate direct light; initially their bounce is omitted rather than left behind as stale illumination.

Reduce global ambient to a small floor. Spatial sky visibility must also gate outdoor diffuse/specular environment light in caves. Adding a dark probe does not subtract an existing bright global contribution. If native probe behavior cannot meet entrance and leakage tests, add one shared spatial-environment shader hook for all material families; this is an explicit integration milestone. Do not paper over it with room-wide unshadowed fill.

Approach	Decision for Hither
Clustered forward + bounded shadows	Baseline. Reuses the current engine and solves item direct lighting.
Full deferred rewrite	Do not lead with it. Needs compatible material/depth outputs and a measured advantage over the existing forward path.
Static probes / selective baking	Second stage for stable den ambient/bounce. New runtime generation pipeline required. Whole-world offline baking is a poor fit for new procedural seeds.
Screen-space GI	Optional polish only. Screen-visible data cannot reliably represent all offscreen sources and occluders.
Dynamic probe GI (DDGI)	Best later prototype if moving-light bounce is worth its tracing, visibility-cache and update costs.
Voxel cone tracing	Defer: adds hierarchical voxelization and updates alongside the existing terrain and foliage representations.
Solari / ray-traced lighting	Experimental quality path. Requires ray-query features and a compatible ray-tracing scene/material representation.

Method evidence: Epic SSGI documentation describes supplemental screen-space bounce; Production probe GI research covers probe scheduling and large-world cascades; NVIDIA voxel cone tracing research describes dynamic voxelization; Bevy 0.19.1 Solari source identifies experimental status, feature requirements and StandardMaterial-based setup. These tradeoff rankings are recommendations for Hither, not benchmark results.

Implementation sequence and release gates

Phase	Concrete work	Exit criterion
A. Baseline	Freeze code/assets/settings; add deterministic lighting test poses and diagnostic toggles.	Current images and CPU/GPU timings reproducible without competing game/compiler work.
B. Shared direct light	Add terrain/rock PBR adapter; centralize sun/sky; audit depth and shadow variants.	A moving point light affects every material class; wall and portal tests pass.
C. Item emitters	Add LightEmitter, presets, lifecycle adapter and shared budget; migrate lamps/torches.	Generic test item can be placed, equipped, dropped, disabled and despawned without orphaned lights.
D. Visual calibration	World-space hand lighting, fixed exposure calibration, roughness tuning and optional bloom.	Snow, skin, foliage and caves remain consistent across all camera modes and quality settings.
E. Spatial ambient	Prototype den probes, generation budget and sky-visibility treatment.	Entrance remains bright while deep room is dark; no cross-wall interpolation or stale bounce.
F. Performance gate	Run paired stress cases; choose final caps and optional effects from measurements.	Documented median/p95/p99, images, known limits and verified fallback behavior.

Benchmark cases

Use 0, 32, 128 and 512 candidate emitters, both spread out and overlapping. Above-cap cases test graceful degradation. Separate direct-light and shadow costs. Include a crowded den, moving held torch, wall-blocked lamp, offscreen lamp, forest canopy, snowfield, cave entrance, mountain flight, teleport and repeated load/unload. Test first/third person and spectator at native and reduced resolution, FXAA and supported MSAA modes.

Extend tools/benchmark_render.py and the existing diagnostics. Freeze release binaries and assets, use fixed seeds/poses, warm up streaming, repeat in alternating order, and record all view timings including hands. Keep instrumented GPU runs separate from clean frame-time comparisons. Record cluster occupancy, admitted/evicted lights, shadow faces/draws, CPU allocation spikes and memory after repeated streaming.

Proposed acceptance targets, not predictions

Start by targeting no more than 5% median/p95 frame-time regression in matched no-emitter scenes, and an incremental lighting GPU budget of 2 ms for the Balanced 128-candidate/two-shadow stress case at 1440p on the development GPU. These are provisional engineering targets; establish named target hardware and frame-rate requirements before declaring them product guarantees. Profile 4K separately and at least one less powerful GPU.

Require correct cross-material response, no hard light popping under slow movement, no wall leakage from shadow eviction, stable camera-mode transitions, and bounded memory. Run cargo fmt, strict Clippy, relevant lifecycle/budget tests, existing render/graphics checks, and isolated runtime shader validation. Do not count pre-existing unrelated failures as new lighting regressions or claim a clean suite if they remain.

Estimated relative effort: shared direct lighting and emitter lifecycle are moderate; hand-camera correctness, procedural shadow variants and spatial ambient are the major risks. Do not commit to a calendar estimate until Phase B validates the integration. The first demonstrable milestone is one generic glowing item lighting ground, rock, foliage and character consistently.

Evidence and decision limits

Primary technical references were checked on 7 September 2026. Bevy implementation recommendations are pinned to 0.19.1, matching Cargo.toml and the installed dependency source. Research papers establish methods and limitations; none supplies a performance guarantee for Hither.

Engine integration

Bevy 0.19.1 PBR shader and Bevy 0.19.1 material extension — Bevy contributors, release 0.19.1. Clustered point/spot evaluation, shadow-receiver flags, exposure and procedural-material integration. Local source was inspected as well as the public release source.

Bevy 0.19.1 point lights — Bevy contributors, release 0.19.1. Photometric power, finite range, six shadow-map faces, shared map resolution and experimental soft-shadow caveats.

Bevy 0.19.1 irradiance volumes — Bevy contributors, release 0.19.1. Ambient-cube volumes, required input data, interpolation and platform limitations. Hither would need to supply a procedural baking/generation system.

Bevy 0.19.1 Solari source — Bevy contributors, release 0.19.1. Experimental ray-traced lighting, geometry component and GPU feature requirements. Custom Hither material compatibility remains unproven.

Rendering methods

Filament rendering reference — Romain Guy and Mathias Agopian / Google, living documentation. Reference for physical light units, linear light, material separation and imaging pipeline.

Production probe GI research — Zander Majercik, Adam Marrs, Josef Spjut and Morgan McGuire; submitted 2020, revised June 2021. Describes practical probe GI scheduling, transition management and multiresolution volumes.

Epic SSGI documentation — Epic Games, Unreal Engine 4.27 documentation. Used for the screen-space method’s supplemental role, not as a claim about current Bevy integration.

NVIDIA voxel cone tracing research — Cyril Crassin and coauthors / NVIDIA, 2011. Original hierarchical voxel cone-tracing method; historic performance results are not transferred to this project.

Local evidence and unresolved questions

Inspected rendering setup, custom WGSL, graphics settings, native dependency shaders, terrain/view-distance policy, tree/grass rendering, den lanterns, torch animation, hand cameras, architecture notes and benchmark harnesses. Reviewed docs/goblin-den-tunnel.png and docs/biome-forest.png only as historical visual context.

Remaining empirical questions: target GPU/CPU and FPS; acceptable dark-space readability; actual light overlap in populated dens; cost of local shadows with existing tree proxies; hand-view shadow reuse; procedural-shadow correctness; ambient-volume leakage and generation latency. Resolve these through the phased tests, not guessed universal light counts.

Research stopped after the direct-light integration was verified against release source, major alternatives had primary evidence, and remaining decision gaps required a prototype or hardware measurements. The recommended architecture is high confidence; numeric budgets and the final visual tuning remain provisional.